

THE CRUX: HOW TO IMPLEMENT A SIMPLE FILE SYSTEM

How can we build a simple file system? What structures are needed on the disk? What do they need to track? How are they accessed?

The Way To Think

There are two aspects to a file system.

The first is the data structures of the file system. In other words, what types of on-disk structures are utilized by the file system to organize its data and metadata.

The second is its access methods. How does it map the calls made by a process such as `open()`, `read()`, `write()`, etc onto its structures.

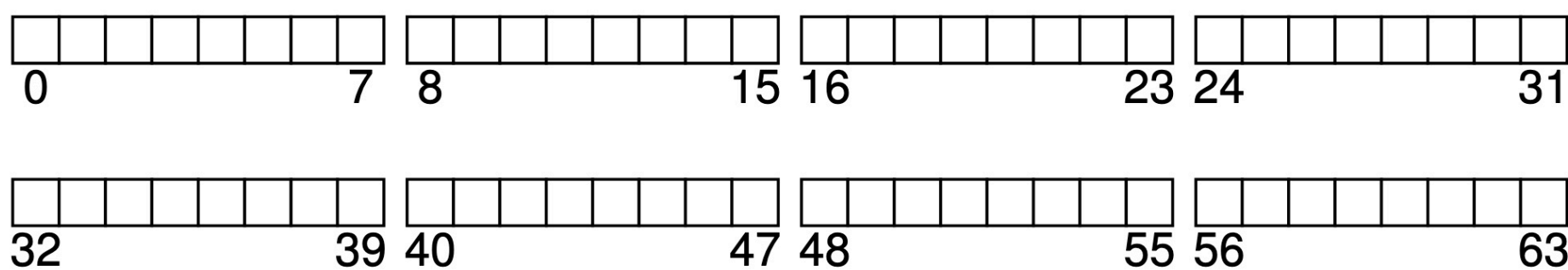
Which structures are read during the execution of a particular system call? Which are written? How efficiently are all these steps performed?

Overall Organization

Overall on disk organization of the data structures of the vofs file system.

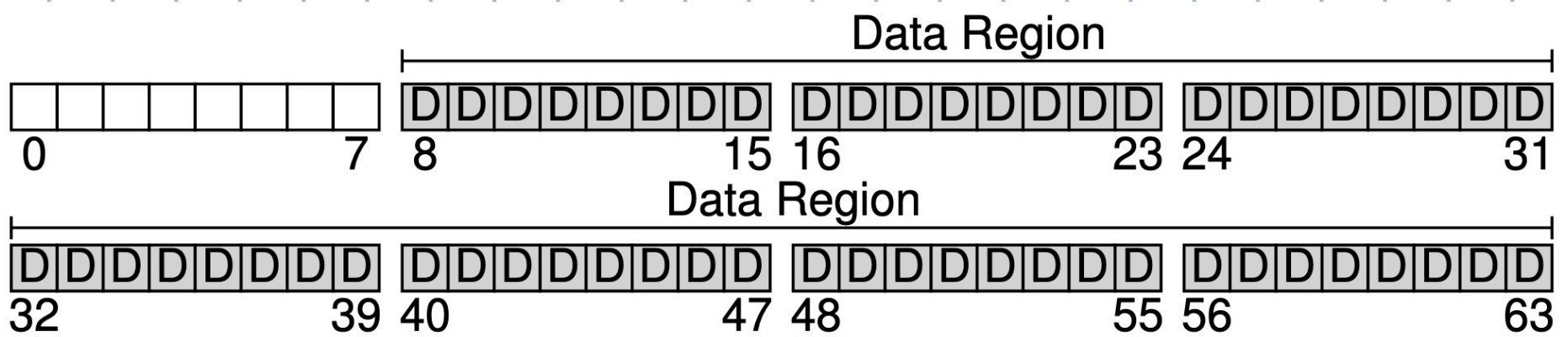
We divide our disk into blocks of 4KB. The blocks are addressed from 0 to $N-1$.

Assume we have a really small disk, with just 64 blocks



Lets call the region of the disk we use for user data the data region, and reserve a fixed portion of the disk for these blocks, say last

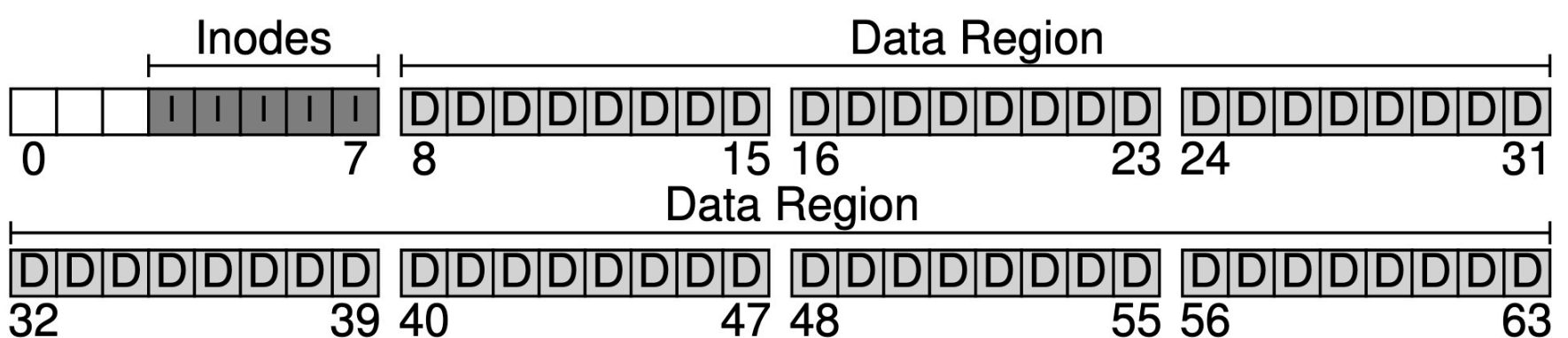
54 blocks:



The file system has to track information about each file (metadata) such as which data blocks in data region comprise a file, the size of the file, its owner and access rights, access and modify times etc.

To store this information, file systems usually have a structure called inode.

We reserve some space on disk for them called inode table.



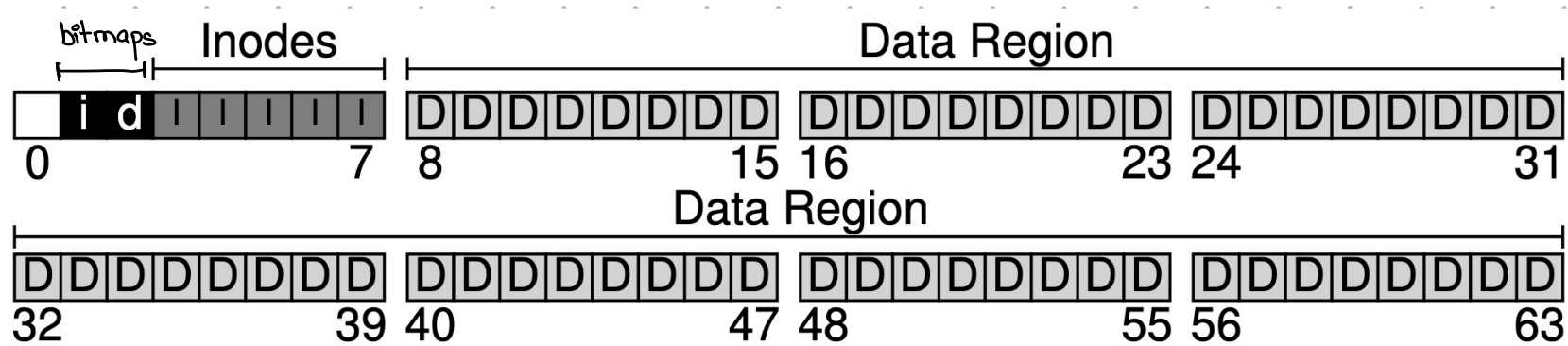
Inodes are typically 128 or 256 bytes. Assuming 256 byte per inode, a 4KB block can hold 16 inodes. Our file system above contains 80 inodes.

We also need a way to track whether inodes or data blocks are free or allocated. Such allocation structures are a requisite element in any file system.

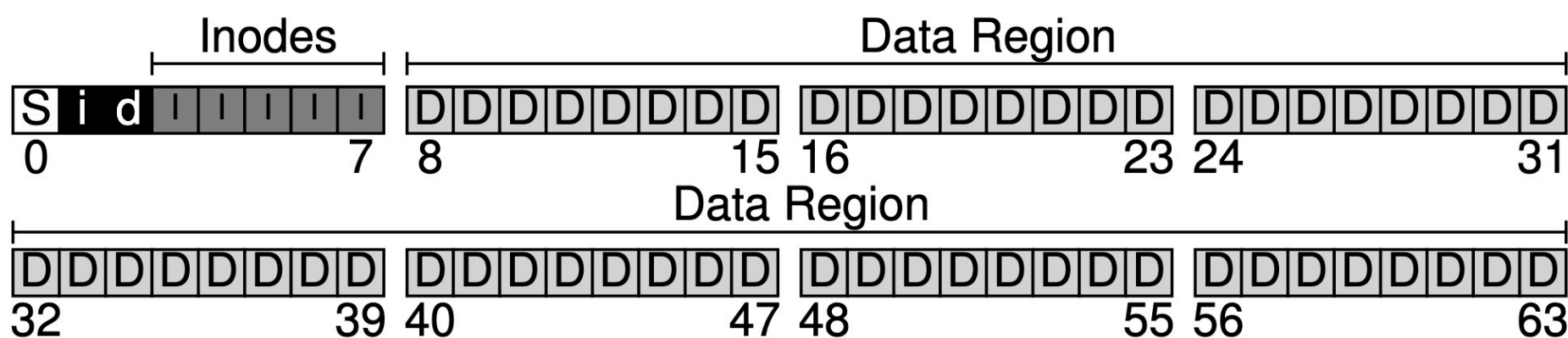
For this, we can use a free list but instead choose a simple structure known as bitmap, one for data region (data bitmap), and one for inode table (inode bitmap).

A bitmap is a simple structure where each bit

is used to indicate whether the corresponding object/block is free (0) or in-use (1).



The final block is reserved for the Superblock S, which contains information about this file system such as how many inodes & data blocks are in the file system, where the inode table begins etc along with a magic number to identify the file system type.



Thus, when mounting a file system, the operating system will first read the superblock first & then attach to volume tree.

File Organization: The Inode

One of the most important on-disk structures of a file system is the inode.

The name is short for index node.

These nodes were originally arranged as an array, and the array indexed into when accessing a particular inode.

Each inode is implicitly referred to by a number (called the i-number or low-level name).

In vsfs, given an i-number, you should directly be able to calculate where on the disk the corresponding inode is located.

For example, consider the inode table above: 20KB in size, consisting of 80 inodes.

Further, assume that the inode region starts at 12KB (super block at 0KB, inode bitmap at 4KB, data bitmap at 8KB):

The Inode Table (Closeup)

				iblock 0				iblock 1				iblock 2				iblock 3				iblock 4			
Super	i-bmap	d-bmap		0	1	2	3	16	17	18	19	32	33	34	35	48	49	50	51	64	65	66	67
				4	5	6	7	20	21	22	23	36	37	38	39	52	53	54	55	68	69	70	71
				8	9	10	11	24	25	26	27	40	41	42	43	56	57	58	59	72	73	74	75
				12	13	14	15	28	29	30	31	44	45	46	47	60	61	62	63	76	77	78	79
0KB	4KB	8KB	12KB	16KB				20KB				24KB				28KB				32KB			

To read inode number 32, the file system would first calculate offset into inode region ($32 \cdot \text{sizeof}(\text{inode})$ or 8192), add it to start address of inode table on disk ($\text{inodeStartAddr} = 12KB$), and get correct byte address of the desired block of inodes: 20KB.

The disks are not byte addressable, but rather consist of a large number of addressable sectors of 512 bytes.

Thus, to fetch the block of inodes that contains inode 32, the file system would issue a read to sector $\frac{20 \times 1024}{512}$, or 40, to fetch the desired inode block.

More generally,

```
blk      = (inumber * sizeof(inode_t)) / blockSize;
sector   = ((blk * blockSize) + inodeStartAddr) / sectorSize;
```

Inode contains all information you would need about a file: its type, its size, the number of blocks allocated to it, protection information, time information, where its data blocks reside on disk. All this information about a file is referred to as metadata

Size	Name	What is this inode field for?
2	mode	can this file be read/written/executed?
2	uid	who owns this file?
4	size	how many bytes are in this file?
4	time	what time was this file last accessed?
4	ctime	what time was this file created?
4	mtime	what time was this file last modified?
4	dtime	what time was this inode deleted?
2	gid	which group does this file belong to?
2	links_count	how many hard links are there to this file?
4	blocks	how many blocks have been allocated to this file?
4	flags	how should ext2 use this inode?
4	osd1	an OS-dependent field
60	block	a set of disk pointers (15 total)
4	generation	file version (used by NFS)
4	file_acl	a new permissions model beyond mode bits
4	dir_acl	called access control lists

Figure 40.1: Simplified Ext2 Inode

One of the most important decisions in the design of the inode is how it refers to where data blocks are.

One approach would be to have one or more direct pointers (disk addresses) inside the inode; each pointer refers to one disk block that belongs to the file.

Such an approach is limited. For example, if you want to have a file that is really big — bigger than the block size multiplied by the number of direct pointers in the inode — you are out of luck! not store kernel

The Multi-Level Index

We have a special pointer known as an indirect pointer.

Instead of pointing to a block that contains user data, it points to a block that contains more pointers, each of which points to user data.

Thus, an inode may have a fixed number of direct pointers (12) & a single indirect pointer.

If a file grows large enough, an indirect block is allocated from the data region of the disk, and the inode's slot for an indirect pointer is set to it.

Assuming 4kB blocks and 4-byte disk addresses, that adds another 1024 pointers; the file can grow to be $12 \times 4\text{kB} + 1024 \times 4\text{kB} = 4144\text{kB}$.

You can support even larger files. To do so, just add another pointer to inode: the double indirect pointer.

This pointer refers to a block that contains pointers to indirect blocks, each of which contain pointers to data blocks.

You go even add triple indirect pointers.

This is referred to as the multi-level index approach.

Let's examine an example with twelve direct pointers, as well as both a single and double indirect block.

Assuming block size of 4kB & 4 byte pointers, this structure can accommodate 4GB.

$$12 \times 4\text{kB} + 1024 \times 4\text{kB} + 1024 \times 1024 \times 4\text{kB}$$

Can you figure out how big of a file can be handled with the addition of a triple-indirect block? (hint: pretty big)

$$12 \times 4\text{kB} + 1024 \times 4\text{kB} + 1024^2 \times 4\text{kB} + 1024^3 \times 4\text{kB}$$

Many filesystems use a multi-level index including Linux ext2 & ext3.

Other filesystems use extents instead of simple pointers.

Why an imbalanced tree like this?

Researchers found that most files are small. Thus small number of direct pointers, an inode can directly point to 48kB of data.

Below is a summary by a study:

Most files are small	~2K is the most common size
Average file size is growing	Almost 200K is the average
Most bytes are stored in large files	A few big files use most of space
File systems contain lots of files	Almost 100K on average
File systems are roughly half full	Even as disks grow, file systems remain ~50% full
Directories are typically small	Many have few entries; most have 20 or fewer

Figure 40.2: File System Measurement Summary

TIP: CONSIDER EXTENT-BASED APPROACHES

A different approach is to use **extents** instead of pointers. An extent is simply a disk pointer plus a length (in blocks); thus, instead of requiring a pointer for every block of a file, all one needs is a pointer and a length to specify the on-disk location of a file. Just a single extent is limiting, as one may have trouble finding a contiguous chunk of on-disk free space when allocating a file. Thus, extent-based file systems often allow for more than one extent, thus giving more freedom to the file system during file allocation.

In comparing the two approaches, pointer-based approaches are the most flexible but use a large amount of metadata per file (particularly for large files). Extent-based approaches are less flexible but more compact; in particular, they work well when there is enough free space on the disk and files can be laid out contiguously (which is the goal for virtually any file allocation policy anyhow).

Directory Organization:

In vsfs, a directory is a list of (entry name, inode number) pairs.

For each file or directory in a given directory, there is a string & a number in the data block(s) of the directory.

For each string, there may also be a length.
For example,

inum	reclen	strlen	name
5	12	2	· → current directory
2	12	3	· · → previous directory
12	12	4	foo
13	12	4	bar
24	36	28	foobar_is_a_pretty_longname

In this, there is also a record length = the total bytes for the name plus any left over space.
(It is essentially size of a entry)

Deleting a file by calling `unlink()` can leave an empty space in the middle of the directory, hence there should some way to mark that as well (eg with a reserved inode number such as zero).

Due to this, `reclen` is used: a new entry may reuse an old, bigger entry.

A directory has an inode, somewhere in the inode table with file type marked as directory, with direct & indirect blocks.

XFS stores directories in B-tree form, making file operation faster instead of this linear list

ASIDE: LINKED-BASED APPROACHES

Another simpler approach in designing inodes is to use a **linked list**. Thus, inside an inode, instead of having multiple pointers, you just need one, to point to the first block of the file. To handle larger files, add another pointer at the end of that data block, and so on, and thus you can support large files.

As you might have guessed, linked file allocation performs poorly for some workloads; think about reading the last block of a file, for example, or just doing random access. Thus, to make linked allocation work better, some systems will keep an in-memory table of link information, instead of storing the next pointers with the data blocks themselves. The table is indexed by the address of a data block D ; the content of an entry is simply D 's next pointer, i.e., the address of the next block in a file which follows D . A null-value could be there too (indicating an end-of-file), or some other marker to indicate that a particular block is free. Having such

a table of next pointers makes it so that a linked allocation scheme can effectively do random file accesses, simply by first scanning through the (in memory) table to find the desired block, and then accessing (on disk) it directly.

Does such a table sound familiar? What we have described is the basic structure of what is known as the **file allocation table**, or **FAT** file system. Yes, this classic old Windows file system, before NTFS [C94], is based on a simple linked-based allocation scheme. There are other differences from a standard UNIX file system too; for example, there are no inodes per se, but rather directory entries which store metadata about a file and refer directly to the first block of said file, which makes creating hard links impossible. See Brouwer [B02] for more of the inelegant details.

Free Space Management

When we create a file, we will have to allocate an inode for that file.

The filesystem will thus search through bitmap for an inode that is free and allocate it to that file and mark the inode as used with a 1 & update the bitmap with correct information.

ext2 & ext3 will look for a sequence of blocks (say 8) that are free when a new file is created and needs data blocks. This guarantees that a portion of the file will be contiguous.

Such a pre-allocation policy is commonly used heuristic.

Access Paths: Reading & Writing

For following examples, let us assume file system has been mounted & thus the superblock is already in memory. Everything else (i.e. inodes, directory) is still on disk.

Reading A File From Disk

Let us assume you want to simply open a file "/foo/bar", read it & then close it.

Further, assume the file is 12KB in size & 3 data

blocks

	data bitmap	inode bitmap	root inode	foo inode	bar inode	root data	foo data	bar data [0]	bar data [1]	bar data [2]
open(bar)			read	read	read	read	read			
read()					read		read			
read()					read			read		
read()					read					
read()					read					read

Figure 40.3: File Read Timeline (Time Increasing Downward)

When you issue an `open("/foo/bar", O_RDONLY)` call, the file system first needs to find the inode for the file `bar`, to obtain some basic information about the file (permissions information, file size, etc.). To do so, the file system must be able to find the inode, but all it has right now is the full pathname. The file system must **traverse** the pathname and thus locate the desired inode.

All traversals begin at the root of the file system, in the **root directory** which is simply called `/`. Thus, the first thing the FS will read from disk is the inode of the root directory. But where is this inode? To find an inode, we must know its i-number. Usually, we find the i-number of a file or directory in its parent directory; the root has no parent (by definition). Thus, the root inode number must be “well known”; the FS must know what it is when the file system is mounted. In most UNIX file systems, the root inode number is 2. Thus, to begin the process, the FS reads in the block that contains inode number 2 (the first inode block).

Once the inode is read in, the FS can look inside of it to find pointers to data blocks, which contain the contents of the root directory. The FS will thus use these on-disk pointers to read through the directory, in this case looking for an entry for `foo`. By reading in one or more directory data blocks, it will find the entry for `foo`; once found, the FS will also have found the inode number of `foo` (say it is 44) which it will need next.

The next step is to recursively traverse the pathname until the desired inode is found. In this example, the FS reads the block containing the inode of `foo` and then its directory data, finally finding the inode number of `bar`. The final step of `open()` is to read `bar`’s inode into memory; the FS then does a final permissions check, allocates a file descriptor for this process in the per-process open-file table, and returns it to the user.

Once open, the program can then issue a `read()` system call to read from the file. The first read (at offset 0 unless `lseek()` has been called) will thus read in the first block of the file, consulting the inode to find the location of such a block; it may also update the inode with a new last-accessed time. The read will further update the in-memory open file table for this file descriptor, updating the file offset such that the next read will read the second file block, etc.

At some point, the file will be closed. There is much less work to be done here; clearly, the file descriptor should be deallocated, but for now, that is all the FS really needs to do. No disk I/Os take place.

Also, note that the amount of I/O generated by the open is proportional to the length of the path name.

Writing A File To Disk

	data bitmap	inode bitmap	root inode	foo inode	bar inode	root data	foo data	bar data [0]	bar data [1]	bar data [2]
create (/foo/bar)		read write	read	read	read write	read	read	write		
write()	read write				read write			write		
write()	read write				write read				write	
write()	read write				read write					write

Figure 40.4: File Creation Timeline (Time Increasing Downward)

Writing to a file is a similar process. First, file must be opened. Then, the application can issue

write() calls to update the file with new contents. Finally, the file is closed.

Unlike reading, writing to the file may also allocate a block (unless the block is being overwritten) and

Thus, each write not only has to write data to disk but to first decide which block to allocate to the file & thus update other structures of the disk accordingly.

Thus, traffic is even worse.

In above figure, there are 10 I/Os to create the file. Further, each write to file takes 5 I/Os.

THE CRUX: HOW TO REDUCE FILE SYSTEM I/O COSTS

Even the simplest of operations like opening, reading, or writing a file incurs a huge number of I/O operations, scattered over the disk. What can a file system do to reduce the high costs of doing so many I/Os?

Caching and Buffering

As seen above, reading & writing can be expensive.

To remedy this, most file systems use system memory to cache important blocks.

Without caching, every file open would require at least 2 reads for every level in the directory hierarchy (one to read the inode of the directory in question and at least one to read its data).

Early file systems introduced fixed-size cache about 10% of total memory allocated at boot time to hold popular blocks. This is called static partitioning.

Strategies such as LRU & different variants would decide which blocks to keep in cache.

Modern systems use dynamic partitioning. Specifically, many modern operating systems integrate virtual

memory pages & file system pages into a unified page cache.

In this way, memory can be allocated more flexibly across virtual memory & file system, depending on which needs more.

Now, imagine the file open example with caching. The first open may generate a lot of I/O traffic to read in directory inode and data, but subsequent file opens of that same file (or files in same directory) will mostly hit in cache.

Let us also consider the effect of caching on writes. Whereas read I/O can be avoided altogether with a sufficiently large cache, write traffic has to go to disk in order to become persistent. Thus, a cache does not serve as the same kind of filter on write traffic that it does for reads. That said, **write buffering** (as it is sometimes called) certainly has a number of performance benefits. First, by delaying writes, the file system can **batch** some updates into a smaller set of I/Os; for example, if an inode bitmap is updated when one file is created and then updated moments later as another file is created, the file system saves an I/O by delaying the write after the first update. Second, by buffering a number of writes in memory, the system can then **schedule** the subsequent I/Os and thus increase performance. Finally, some writes are avoided altogether by delaying them; for example, if an application creates a file and then deletes it, delaying the writes to reflect the file creation to disk **avoids** them entirely. In this case, laziness (in writing blocks to disk) is a virtue.

For the reasons above, most modern file systems buffer writes in memory for anywhere between five and thirty seconds, representing yet another trade-off: if the system crashes before the updates have been propagated to disk, the updates are lost; however, by keeping writes in memory longer, performance can be improved by batching, scheduling, and even avoiding writes.

Some applications (such as DBs) don't enjoy this tradeoff.

They simply force writes to disk by calling `fsync()`, by using direct I/O interfaces that work around cache, or by using raw disk interfaces that work around the cache.

TIP: UNDERSTAND STATIC VS. DYNAMIC PARTITIONING

When dividing a resource among different clients/users, you can use either **static partitioning** or **dynamic partitioning**. The static approach simply divides the resource into fixed proportions once; for example, if there are two possible users of memory, you can give some fixed fraction of memory to one user, and the rest to the other. The dynamic approach is more flexible, giving out differing amounts of the resource over time; for example, one user may get a higher percentage of disk bandwidth for a period of time, but then later, the system may switch and decide to give a different user a larger fraction of available disk bandwidth.

Each approach has its advantages. Static partitioning ensures each user receives some share of the resource, usually delivers more predictable performance, and is often easier to implement. Dynamic partitioning can achieve better utilization (by letting resource-hungry users consume otherwise idle resources), but can be more complex to implement, and can lead to worse performance for users whose idle resources get consumed by others and then take a long time to reclaim when needed. As is often the case, there is no best method; rather, you should think about the problem at hand and decide which approach is most suitable. Indeed, shouldn't you always be doing that?

TIP: UNDERSTAND THE DURABILITY/PERFORMANCE TRADE-OFF

Storage systems often present a durability/performance trade-off to users. If the user wishes data that is written to be immediately durable, the system must go through the full effort of committing the newly-written data to disk, and thus the write is slow (but safe). However, if the user can tolerate the loss of a little data, the system can buffer writes in memory for some time and write them later to the disk (in the background). Doing so makes writes appear to complete quickly, thus improving perceived performance; however, if a crash occurs, writes not yet committed to disk will be lost, and hence the trade-off. To understand how to make this trade-off properly, it is best to understand what the application using the storage system requires; for example, while it may be tolerable to lose the last few images downloaded by your web browser, losing part of a database transaction that is adding money to your bank account may be less tolerable. Unless you're rich, of course; in that case, why do you care so much about hoarding every last penny?